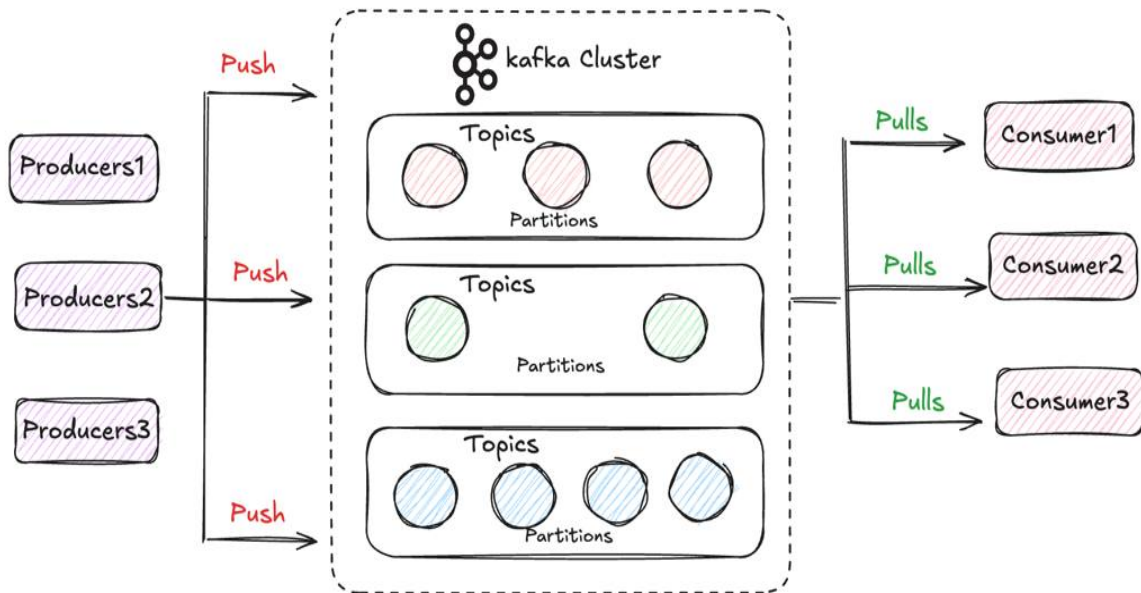


Kafka Fundamentals

Kafka is a **distributed, partitioned, append-only commit** log designed as a highly available, horizontally scalable storage system. There are certain terms associated with the Kafka and concepts associated with it which we will see as we grow, but below is the very high-level understanding of the Kafka internals.



Sequential Log Architecture 🍪: Kafka writes messages sequentially to disk files called **segments**. Sequential disk access is orders of magnitude faster than random access, approaching memory speeds. It leverages the OS page cache heavily and utilizes Zero-Copy operations (sendfile system call) to bypass user-space memory buffers, copying data directly from the page cache to the network socket.

Partitions as the Unit of Scalability 🍷: A topic is divided into partitions, which are distributed across a cluster of brokers. Partitions are the atomic unit of parallelism. Only one consumer in a specific consumer group can read from a single partition at a time, ensuring strict sequential ordering of events within that partition.

Decoupled State Management 📌: In Kafka, the broker is intentionally "dumb" and stateless regarding consumer progress. **Consumers track their own position using an integer pointer called an Offset**. This shift moves the computational overhead of state tracking from the cluster to the edge clients, allowing Kafka to scale linearly to millions of messages per second.

Consensus & Replication 👤: Kafka guarantees high availability through partitioned replication. **Each partition has one leader and multiple followers**. Write operations go through the leader and are replicated to In-Sync Replicas (ISR). Modern Kafka clusters utilize KRaft (Kafka Raft metadata mode) for internal consensus, removing the operational bottleneck of Apache ZooKeeper.

BootStrap Server

In Apache Kafka, a **bootstrap server** is the initial point of contact that a Kafka client (producer or consumer) uses to discover the full topology of the Kafka cluster. 🌐

When a client starts up, it doesn't need to know the address of every single broker in the cluster. Instead, it connects to the designated bootstrap server(s) and issues a Metadata Request 📄.

Here is exactly what happens during that handshake:

Initial Connection 📡: The client connects to a broker listed in its bootstrap.servers configuration.

Metadata Exchange 🔄: The bootstrap broker responds with the cluster's current metadata layout. This includes the full list of active brokers, all topics, their partitions, and exactly which broker is currently acting as the leader for each partition.

Direct Routing 🚀: The client caches this metadata locally. From that point forward, it establishes direct TCP connections to the specific brokers holding the partition leaders it needs to read from or write to.

Offsets

An **offset** in Kafka is a **sequential, incremental integer used to uniquely identify messages within a specific partition**. It acts as a continuous "bookmark" or cursor, allowing producers, consumers, and brokers to track the exact position of data streams and ensure messages are read in order.

How Offsets Work

Immutability: Each message added to a partition is assigned the next available integer (starting at 0 for the first message).

Partition-Specific: Offsets are only meaningful within a specific partition. For example, Offset 5 in Partition 0 represents entirely different data than Offset 5 in Partition 1.

Ordering: Kafka guarantees the order of messages only within a single partition, tracked by these offsets

Types of Offsets

Current Offset: Tracks the most recent message Kafka has provided to a consumer application.

Committed Offset: The last message a consumer group has successfully processed. If a consumer restarts or crashes, it uses the committed offset to resume reading where it left off.

Log-End Offset: The offset of the very last message appended to a partition by a producer.

Consumer Lag : Lag is the difference between the Log-End Offset and the Current Offset. It tells you how far behind a consumer is from the messages being produced. Consumer offsets are stored in an internal Kafka topic named `__consumer_offsets` to ensure state tolerance.